

University Name

Bachelor Thesis

Benchmarking Large Language Models on Algorithmic Programming Tasks

A Case Study Using Advent of Code with Python and Rust

Your Name

Supervisor: Prof. Dr. Name

Degree Program: Computer Science

Submission Date: April 5, 2026

Abstract

This thesis evaluates the performance of large language models on algorithmic programming tasks using Advent of Code problems. The generated solutions are compared across Python and Rust with respect to correctness, runtime, memory usage, compile success, and code quality. The goal is to assess both the algorithmic reasoning ability of the models and the influence of programming language on generated code quality. To ensure fairness and reproducibility, all experiments are conducted under controlled conditions using standardized prompts, fixed problem sets, and automated evaluation scripts.

Acknowledgements

Optional.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

Large language models are increasingly used for software development tasks, including code generation, debugging, refactoring, and explanation. As these systems become more common in practice, it becomes necessary to evaluate not only whether they can produce syntactically correct code, but also whether they can solve algorithmic problems reliably and efficiently.

Algorithmic programming tasks are particularly suitable for this kind of evaluation because they require structured reasoning, careful input handling, and awareness of computational complexity. In contrast to simple code completion, such tasks test whether a model can derive an algorithmic solution from a textual specification and implement it correctly.

1.2 Problem Statement

Although large language models can often generate plausible code, their actual performance on algorithmically demanding tasks remains uncertain. It is therefore important to investigate:

- how often they generate correct solutions,
- whether they select efficient algorithms,
- how robust their outputs are across different programming languages, and
- what kinds of errors occur in practice.

This thesis addresses these questions using Advent of Code as the evaluation benchmark and comparing generated solutions in Python and Rust.

1.3 Research Objectives

The main objective of this thesis is to benchmark selected large language models on algorithmic programming tasks and evaluate their generated solutions systematically. The study focuses on:

- correctness,
- runtime performance,
- memory usage,
- compile success,
- and structural characteristics of the generated code.

1.4 Research Questions

The main research question is:

How effectively can large language models solve algorithmic programming tasks from Advent of Code, and how do their generated solutions differ between Python and Rust?

Sub-questions include:

- What proportion of selected tasks can be solved correctly?
- How do runtime and memory usage differ between Python and Rust solutions?
- Which categories of errors occur most frequently?
- Does the target programming language influence correctness, compile success, or code quality?

1.5 Structure of the Thesis

Chapter 2 introduces the theoretical background. Chapter 3 discusses related work. Chapter 4 explains the methodology. Chapter 5 presents the implementation and experimental setup. Chapter 6 reports the benchmark results. Chapter 7 discusses the findings and limitations. Chapter 8 concludes the thesis and outlines future work.

Chapter 2

Background

2.1 Large Language Models

Large language models are neural networks trained on large text corpora using the transformer architecture. Recent models are capable of generating code, explaining algorithms, and assisting with programming tasks. Their coding ability is often attributed to exposure to large-scale code repositories and documentation during training **chen2021codex**, **openai2023gpt4**.

In the context of this thesis, large language models are treated as code-generating systems that receive a natural-language problem description and produce a complete solution in a specified programming language.

2.2 Algorithmic Programming Tasks

Algorithmic programming tasks require more than local code generation. They usually involve:

- parsing structured input,
- identifying the underlying computational problem,
- designing a suitable algorithm,
- implementing the solution correctly,
- and reasoning about complexity.

Typical categories include graph problems, dynamic programming, simulation, greedy methods, search, and string processing.

2.3 Advent of Code

Advent of Code is an annual programming challenge consisting of 25 daily problems released each December **aoc**. Each problem has two parts, with the second part usually extending or modifying the first. The problems span a wide range of algorithmic categories and difficulty levels, which makes Advent of Code a useful benchmark for evaluating the problem-solving ability of code generation systems.

Compared to benchmark suites focused on short function synthesis, Advent of Code problems often require full-program solutions, input parsing, and multi-step reasoning. This makes them particularly suitable for evaluating practical algorithmic problem solving.

2.4 Programming Languages: Python and Rust

This thesis compares generated solutions in Python and Rust.

2.4.1 Python

Python is widely used for rapid prototyping and algorithmic problem solving. Its concise syntax and high-level data structures make it suitable for expressing algorithms clearly and with relatively little boilerplate. However, Python generally has lower runtime performance and higher memory overhead than lower-level languages.

2.4.2 Rust

Rust is a systems programming language designed for performance and memory safety. It offers efficient execution and strong compile-time guarantees, but its strict ownership and borrowing rules can make code generation more challenging. This makes Rust an interesting comparison language: it may reveal whether large language models struggle more when the target language imposes stronger structural constraints.

Chapter 3

Related Work

3.1 LLMs for Code Generation

Prior work has shown that large language models can generate working code for a variety of programming tasks. Codex and related systems were evaluated on benchmarks such as HumanEval, which focus mainly on functional correctness **chen2021codex**. Further work examined program synthesis capabilities on tasks such as MBPP and similar datasets **austin2021programsynthesis**.

3.2 Benchmarking in Software Engineering

Software engineering benchmarks emphasize reproducibility, fairness, and clear evaluation criteria. These principles are especially important when comparing AI-based systems because small differences in prompts, runtime environment, or manual intervention can strongly affect the outcome.

A proper benchmark should therefore:

- define variables clearly,
- fix the dataset in advance,
- standardize prompts,
- log all runs,
- avoid manual corrections during measurement,
- and report both successes and failures.

3.3 Research Gap

Much prior work evaluates code generation on short benchmark tasks, but fewer studies examine full-program algorithmic challenges such as Advent of Code while simultaneously comparing multiple target languages. This thesis contributes by combining:

- algorithmic programming tasks,
- model comparison,
- and a cross-language evaluation between Python and Rust.

Chapter 4

Methodology

4.1 Study Design

This thesis follows an empirical benchmarking approach. The benchmark compares two code-generating systems across two programming languages on a fixed set of Advent of Code problems.

The independent variables are:

- model,
- target programming language,
- and problem difficulty.

The dependent variables are:

- correctness,
- compile success,
- runtime,
- memory usage,
- and lines of code.

4.2 Scientific Benchmarking Principles

To ensure that the benchmark is scientifically meaningful, the following principles are applied:

4.2.1 Reproducibility

All experiments are executed using the same machine, operating system, prompt templates, and evaluation scripts. The benchmark repository contains the selected problems, raw prompts, generated code, logs, and result tables.

4.2.2 Fairness

All models receive the same prompt structure. Only the problem file, input file, and target language are changed between runs. No model-specific hints are added.

4.2.3 Objectivity

All code is evaluated automatically. Generated solutions are run exactly as produced, without manual fixes before scoring. If repair attempts are included, they are handled using a standardized repair prompt and logged separately.

4.2.4 Transparency

Every run stores:

- the exact prompt,
- the model name,
- the generated output,
- compile or runtime logs,
- and the measured metrics.

4.3 Problem Selection

A fixed set of Advent of Code problems is selected before running the experiments. The selection should include multiple difficulty levels and problem categories, for example:

- parsing and string processing,
- simulation,
- graph traversal,
- state tracking,
- and optimization or dynamic programming.

A suitable benchmark size for a bachelor thesis is 10 to 12 problems. This creates enough data for comparison without making the experiment unmanageable.

4.4 Models and Languages

The benchmark compares two code-generation systems:

- Codex,
- Sonnet.

Each model is evaluated in:

- Python,
- Rust.

This yields the following experimental matrix:

Table 4.1: Experimental matrix

Model	Language
Codex	Python
Codex	Rust
Sonnet	Python
Sonnet	Rust

If 10 problems are used, this results in 40 primary runs before any repair attempts are considered.

4.5 Prompt Design

A standardized prompt template is used across all runs. Only the problem file, input file, and target language are changed. The benchmark prompt is intentionally short, explicit, and easy to automate.

```
1 You are participating in a controlled benchmark for code
   generation.
2
3 Solve the programming problem described in:
4 {PROBLEM_FILE}
5
6 Requirements:
```

```

7 - Language: {LANGUAGE}
8 - Read input from: {INPUT_FILE}
9 - No external libraries
10 - Handle full input correctly (not just examples)
11
12 Output EXACTLY:
13 Part 1: <answer>
14 Part 2: <answer>
15
16 Rules:
17 - Output ONLY the complete source code
18 - No explanations or markdown formatting
19 - Ensure the solution is correct and reasonably efficient
20 - Follow idiomatic style for the chosen language
21
22 Generate the solution now.

```

4.6 Repair Prompt

If the experimental design includes one repair attempt after failure, the following standardized repair prompt can be used:

```

1 The previous solution failed.
2
3 Error:
4 {ERROR_MESSAGE}
5
6 Fix the solution.
7
8 Requirements:
9 - Same input/output format
10 - Output ONLY corrected source code
11 - No explanations or markdown
12
13 Provide the full corrected program.

```

4.7 Evaluation Metrics

The following metrics are collected for each run.

4.7.1 Correctness

Correctness is measured by comparing the program output to the expected output. A run is considered correct only if both parts match exactly.

4.7.2 Compile Success

For Rust, compile success is recorded as a binary metric. This is especially important because syntactically plausible Rust code may still fail due to type, ownership, or borrowing issues.

4.7.3 Runtime

Runtime is measured over repeated runs. Median runtime is used instead of a single run in order to reduce noise from background system activity or scheduling variation.

4.7.4 Memory Usage

Memory usage is measured using maximum resident set size or an equivalent system-level metric.

4.7.5 Lines of Code

Lines of code are collected as a simple structural indicator of code size and verbosity.

4.8 Execution Procedure

Each experiment follows the same pipeline:

1. Submit the prompt to the selected model.
2. Save the raw generated code.
3. Compile the solution if required.
4. Execute the program on the fixed input.
5. Record output, runtime, memory usage, and errors.
6. Compare the output to the expected results.

4.9 Failure Rules

The following outcomes are defined before running the benchmark:

Table 4.2: Failure rules

Case	Interpretation
Compile fails	failure
Runtime error	failure
Timeout	failure
Wrong output	incorrect
Output format incorrect	failure

4.10 Threats to Validity

Several factors may threaten the validity of the results:

- prompt sensitivity,
- model updates over time,
- limited benchmark size,
- and the fact that Advent of Code covers only one style of programming task.

These limitations are discussed again in Chapter 7.

Chapter 5

Implementation

5.1 Benchmark Environment

All experiments are executed on the same hardware and software environment. The thesis should document:

- CPU,
- RAM,
- operating system,
- Python version,
- Rust version,
- and any benchmark tools used.

5.2 Repository Structure

A reproducible repository structure helps keep the benchmark organized. One possible structure is shown below.

```
1 aoc-llm-benchmark/  
2 |-- prompts/  
3 |-- problems/  
4 |-- generated/  
5 |   |-- codex/  
6 |   |   |-- python/  
7 |   |   '-- rust/  
8 |   '-- sonnet/  
9 |       |-- python/  
10 |       '-- rust/
```

```
11 | -- runner/  
12 | -- results/  
13 | -- thesis_notes/
```

5.3 CLI-Based Generation Workflow

The benchmark uses the command-line interfaces of both systems so that all runs can be scripted and logged consistently. The main idea is:

1. provide the same prompt structure to each system,
2. save the returned code to the proper output file,
3. compile and run the solution automatically,
4. and store all logs in a results directory.

5.4 Controlled Benchmarking Workflow

A proper run should follow this order:

1. clean output folders,
2. generate code,
3. save raw outputs,
4. compile or execute,
5. collect metrics,
6. store result tables,
7. commit the state or tag the run.

5.5 Validation and Measurement

Correctness is validated against expected outputs stored for each Advent of Code problem. Runtime is measured using repeated executions and summarized using the median value. Memory usage is collected using system-level resource measurement tools. Lines of code are counted directly from the generated source files.

5.6 Example Data Collection Table

A structured CSV or JSON file should record at least the following fields:

Table 5.1: Example benchmark data schema

Problem	Model	Language	Correct	Runtime	Memory	LOC
Day01	Codex	Python	1	0.12	15 MB	45
Day01	Codex	Rust	1	0.03	6 MB	62
Day01	Sonnet	Python	0	0.18	17 MB	48
Day01	Sonnet	Rust	0	—	—	58

5.7 Why This Setup Is Scientific

This implementation strategy supports scientific evaluation because it:

- minimizes manual intervention,
- preserves raw outputs,
- makes all runs repeatable,
- and allows direct comparison across models and languages.

Chapter 6

Results

6.1 Overview

This chapter presents the benchmark results for correctness, runtime, memory usage, compile success, and code size. The exact values in the example tables below are placeholders and should be replaced with actual benchmark measurements.

6.2 Correctness

Correctness rates can be summarized by model and language.

Table 6.1: Correctness rates by model and language

Model	Python	Rust
Codex	0.00	0.00
Sonnet	0.00	0.00

The interpretation of this table should explain whether one model achieved higher success rates overall and whether the target language had a noticeable effect on correctness.

6.3 Compile Success

Compile success is especially relevant for Rust.

Table 6.2: Compile success rates for Rust solutions

Model	Compile Success Rate
Codex	0.00
Sonnet	0.00

6.4 Runtime

Runtime measurements should be reported using repeated executions and median values.

Table 6.3: Example runtime results

Problem	Model	Language	Median Runtime (s)
Day 1	Codex	Python	0.00
Day 1	Codex	Rust	0.00
Day 1	Sonnet	Python	0.00
Day 1	Sonnet	Rust	0.00

6.5 Memory Usage

Memory usage should be summarized in a similar way, for example using peak resident memory.

Table 6.4: Example memory usage results

Problem	Model	Language	Peak Memory (MB)
Day 1	Codex	Python	0.00
Day 1	Codex	Rust	0.00
Day 1	Sonnet	Python	0.00
Day 1	Sonnet	Rust	0.00

6.6 Code Size

Lines of code can be used as a simple structural measure.

Table 6.5: Example code size results

Problem	Model	Language	LOC
Day 1	Codex	Python	0
Day 1	Codex	Rust	0
Day 1	Sonnet	Python	0
Day 1	Sonnet	Rust	0

6.7 Error Categories

It is useful to group failures into qualitative categories. Possible categories include:

- parsing failures,
- logic errors,
- output formatting errors,
- inefficient algorithms,
- and Rust-specific compile failures.

6.8 Figures

You can also include plots. Example figure placement:

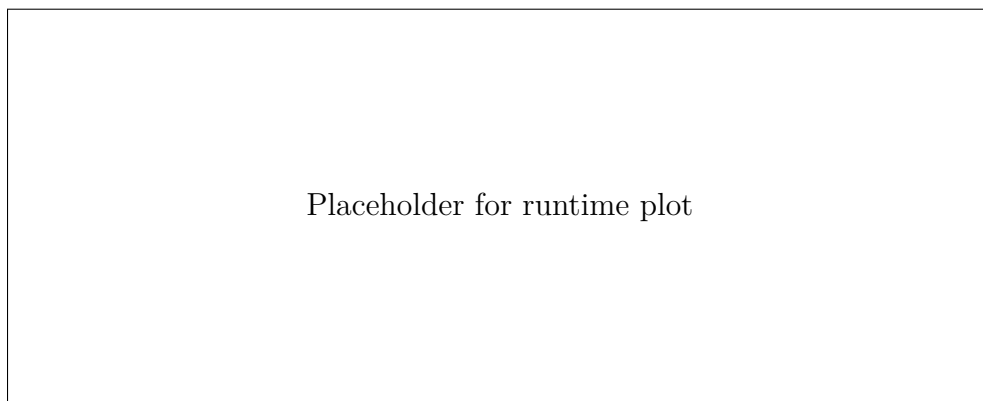


Figure 6.1: Median runtime by model and language

Chapter 7

Discussion

7.1 Interpretation of Results

The discussion chapter should interpret the measured results rather than only repeating tables. Typical questions include:

- Which model solved more tasks correctly?
- Did one model perform better on specific problem categories?
- Were Python solutions more reliable than Rust solutions?
- Did runtime advantages of Rust appear consistently?

7.2 Model Comparison

The comparison between Codex and Sonnet should consider both quantitative and qualitative outcomes. One model may perform better in correctness while the other may generate more concise or more idiomatic code.

7.3 Language Comparison

A likely finding is that Python solutions are more concise and more likely to run on the first attempt, whereas Rust solutions may offer better runtime performance when they compile successfully. This would suggest that the target language influences not only execution characteristics but also the difficulty of generation itself.

7.4 Error Analysis

Error analysis is one of the strongest parts of an empirical thesis. Instead of reporting only aggregate success rates, the thesis should classify failures and discuss patterns. Examples:

- misunderstanding of the second part of the puzzle,
- hidden assumptions based on the example input,
- brute-force approaches that do not scale,
- and language-specific issues such as ownership problems in Rust.

7.5 Limitations

This study has several limitations:

- only a limited number of Advent of Code tasks are included,
- models may change over time,
- prompt phrasing may still influence results,
- and the benchmark focuses on one class of programming problems.

7.6 Implications

The findings can inform both academic research and practical tool usage. If large language models perform well on simpler algorithmic tasks but struggle on more complex stateful or optimization-heavy problems, this suggests that they can be useful assistants but should not be treated as fully reliable autonomous problem solvers.

Chapter 8

Conclusion

8.1 Summary

This thesis benchmarked large language models on Advent of Code tasks and compared generated Python and Rust solutions. The evaluation focused on correctness, compile success, runtime, memory usage, and code size under controlled conditions.

8.2 Answer to the Research Question

The study is designed to determine how effectively large language models solve algorithmic programming tasks and how their behavior differs between Python and Rust. The final answer should summarize the measured differences in correctness, efficiency, and reliability across both model and language.

8.3 Future Work

Future work could extend the benchmark in several directions:

- evaluating more models,
- increasing the number of Advent of Code tasks,
- including additional target languages,
- studying multi-turn repair settings,
- or comparing generated code to human-written baseline solutions.

Appendix A

Appendix

A.1 Complete Benchmark Prompt

```
1 You are participating in a controlled benchmark for code
   generation.
2
3 Solve the programming problem described in:
4 {PROBLEM_FILE}
5
6 Requirements:
7 - Language: {LANGUAGE}
8 - Read input from: {INPUT_FILE}
9 - No external libraries
10 - Handle full input correctly (not just examples)
11
12 Output EXACTLY:
13 Part 1: <answer>
14 Part 2: <answer>
15
16 Rules:
17 - Output ONLY the complete source code
18 - No explanations or markdown formatting
19 - Ensure the solution is correct and reasonably efficient
20 - Follow idiomatic style for the chosen language
21
22 Generate the solution now.
```

A.2 Complete Repair Prompt

```
1 The previous solution failed.
2
3 Error:
4 {ERROR_MESSAGE}
5
6 Fix the solution.
7
8 Requirements:
9 - Same input/output format
10 - Output ONLY corrected source code
11 - No explanations or markdown
12
13 Provide the full corrected program.
```

A.3 Guidelines for Proper and Scientific Benchmarking

For the written thesis, the methodology chapter should make the following points explicit:

- The benchmark is an empirical study.
- Independent variables are model and language.
- Dependent variables are correctness, runtime, memory usage, compile success, and LOC.
- Controlled variables include prompt template, hardware, software environment, and problem set.
- No manual edits are applied before scoring.
- Failures are logged and reported, not hidden.
- Runtime is measured across repeated runs and summarized using medians.

A.4 Suggested Page Distribution

A reasonable page distribution for a thesis of about 35 pages is:

Chapter	Suggested Length
Introduction	3–4 pages
Background	5–6 pages
Related Work	4–5 pages
Methodology	6–7 pages
Implementation	4–5 pages
Results	8–10 pages
Discussion	4–5 pages
Conclusion	2–3 pages